

A Case Study of a Logic Simulation Environment Using *pcieVHost* and Third Party PCIe IP



Simon Southwell

February 2026

Preface

This document is a PDF version that brings together two articles written in February 2026, and uploaded to LinkedIn, being a case study of driving third party PCIe IP with the pcieVHost co-simulation PCIe C model.

Simon Southwell
Cambridge, UK
23rd February 2026

© 2026 Simon Southwell. All rights reserved.

All text and images are copyright of the author. No part of this article may reproduced, stored in a retrieval system, or transmitted in any form or by any means electronic, mechanical, photocopying, recording or otherwise, without prior written permission of the author.

Contact info@anita-simulators.org.uk for any queries.

Contents

PART 1: THIRD PARTY PCIE IP	4
INTRODUCTION.....	4
<i>Purpose and Goals.....</i>	5
CHOOSING 3 RD PARTY PCIE IP	7
ALTERA CYCLONE V HARD IP FOR PCI EXPRESS	7
<i>EDA Tools</i>	7
<i>Generation of Simulation Model</i>	8
Platform Designer and IP Generation	9
Simulation Source Generation	14
WRAPPING THE ALTERA IP	15
<i>Setting of Altera IP Parameters.....</i>	16
<i>Extraction of Required Signals.....</i>	17
<i>PHY Reset Sequencer.....</i>	20
COMPILING THE IP SOURCE	21
CONCLUSIONS	22
PART 2: THE PCIEVHOST TEST BENCH.....	23
INTRODUCTION.....	23
THE HDL TEST BENCH ARCHITECTURE	23
<i>The pcieVHost and mem_model VIP</i>	25
THE PCIEVHOST TEST PROGRAM	27
<i>Received Completion Callback Function.....</i>	28
<i>Thread Safety</i>	29
<i>TLP Test Code</i>	30
BUILDING AND RUNNING THE TEST.....	31
<i>Compiling and Running with the Make File.....</i>	32
<i>Configuring the pcieVHost Link Display Output</i>	32
<i>Results</i>	34
CONCLUSIONS	36
<i>Further Reading.....</i>	36
Co-simulation VIP References	36
Articles	36
External References	37

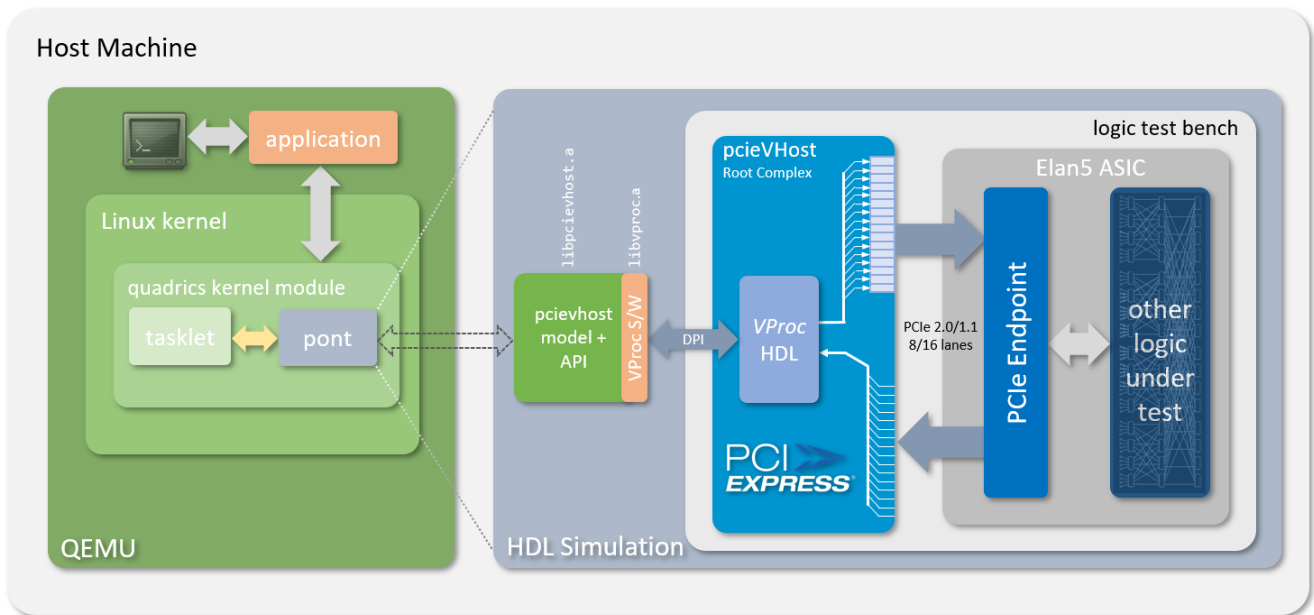
Part 1: Third Party PCIe IP

Introduction

As I write, the first lines of the [pcieVHost](#) model were laid down just about 20 years ago. In the [article](#) I wrote on the internal architecture of the PCIe C model, I summarised the origins of *pcieVHost* as a means to teach myself the PCIe specification as I had been tasked to implement a 16 lane Endpoint design (amongst my other objectives) as *"the available 3rd party implementations had unacceptably high latencies"*, and we were designing supercomputer clusters at the time, so communications between nodes had to be bleeding edge fast. I can't speak for others, but to learn complex concepts I need to implement and try things to reinforce what I'm learning and I started to write C code, as a home side project, to construct the bits for all the different types of traffic that can occur on a PCIe link, initially sending this to a file. To verify that I'd created the patterns I thought were correct, I wrote more code to read the data in the files and decode the patterns and do a rough display of the different patterns of the decoded bits.

The EP design I was implementing was originally meant to be verified against commercial sign-off quality VIP but it became clear, due to licencing costs, that this would not be available until the last stages of development, therefore I needed a means to drive the IP to verify its implementation against what I believed was correct behaviour—it would be verified against the specification using the VIP down the line before committing to tape out. Now my [VProc virtual processor](#) co-simulation element (conceived a few years earlier) was already being used in verifying sub-systems, substituting for multiple "thread" processors and it occurred to me to use this to adapt my crude PCIe C code to drive and decode signals on a PCIe link in a logic simulation. I needed to adapt the way *VProc* was used to do this, and I've written about this, and more generally about modelling arbitrary protocols, in another [article](#). Thus was *pcieVHost* born.

In fact, beyond the PCIe Endpoint test environment, the software team took it and, along with using [Verilator](#) as a free alternative to the commercial licenced simulator we were using, connected this up to QEMU running their Linux drivers and drive a network simulation, through the PCIe interface, to do true co-development of application software and RTL. The diagram below summarises this system, though it is from memory, as I do not have access to any of the documentation I made at that time.



So, the *pcieVHost* model has been used to verify commercial PCIe designed (even if just my own implementations) which, themselves, have been through sign-off testing for PCIe compliance.

Purpose and Goals

Unfortunately, as for the documentation, I do not have access to any of the test environments and IP from the EP development time and so, over the two parts of this document, I want to explore using *pcieVHost* to drive third-party commercially available IP in a way that reflects a real development test environment. Instead of verifying the PCIe implementation (likely signed-off for compliance already), I want to look, instead, at driving available PCIe Hard Macro IP that is provided in some commercial FPGAs that would then drive the application IP being developed that will be using the supplied vendor PCIe implementation. This will serve as a case study, and a template, of what is required to put together such a test environment based around the *pcieVHost* co-simulation model.

For this case study, the general plan is to:

- Uses the FPGA vendor's tools to generate the simulation source code
- Configure the vendor IP suitable for our needs
- Create a wrapper to abstract away the details of the IP's instantiation and create a simplified external interface
- Connect the *pcieVHost* model to the IP's PCIe link

- Use the [Memory Model](#) co-simulation module to stand in for the developer's application RTL
- Go through resetting the IP and bring the link PHY to powered up state (L0)
- Initialise Virtual Channel 0 (VC0) data link layer flow control
- Do a basic configuration space exploration and enumeration
- Transfer, through the PCIe IP, memory data to the memory model and read it back with verification.

This, at first glance, may not look too ambitious as we end up doing a basic couple of memory reads and writes but, due to the nature and complexity of PCIe, an awful lot has to happen to make this possible, and once this point is reached we are communicating with the application IP that's being developed and have arrived at a starting point for it to be exercised to any level of complexity required. For this document, so as not to obfuscate documenting the process of constructing such a test bench environment, I want to keep everything to a bare minimum consistent with reaching the above stated goals. Hopefully, as we proceed, it will be clear what might be required to add more features or complexity, to meet a particular project's needs—i.e. the usual textbook cop out of *"that's left as an exercise for the student"*.

The document is split over two parts and in this first part the focus is on the third-party IP—with a step-by-step process of choosing the IP to use, configuring the IP for requirements with the EDA tools, generating the source code for simulation and creating wrapper logic to configure and abstract away the PCIe IP details, suitable for easy instantiation in a logic simulation environment. It is in the second part that the test bench will be constructed to use this wrapped IP and to run a test from power up to sending memory read and write transactions, through the PCIe block, to attached logic being developed as the application. So it won't be until the second part that we will meet *pcieVHost*, but this first part documents the important process of obtaining the PCIe IP to be used and prepare it for ease of integration into the simulation verification environment.

What's discussed over two parts of the document is all available in the [pcieVHost](#) repository in the [vhdl/testlaltpcie](#) directory. This test bench is for VHDL as there is an extra step in instantiating the Verilog PCIe IP top level in the VHDL for a mixed language simulation, but this just as easily can be done in Verilog or SystemVerilog, and there are versions of *pcieVHost* and *mem_model* in Verilog for use in the languages

Choosing 3rd Party PCIe IP

For this case study, there are two obvious sources of FPGA IP that suggest themselves, and potentially other sources from other vendors as well. I have chosen to use the Altera option, but only because I am most familiar with their Quartus tool chain. The general concepts, if not the details, will be the same for Xilinx and Vivado and even for an ASIC development using 3rd Party PCIe IP for their PCIe interface.

It may be, in the future, I will repeat this processor for Xilinx devices and their tool flow, or maybe someone can volunteer to do this and contribute the advancement of the *pcieVHost* project. Feel free to [contact me](#) directly.

In the rest of this document, for brevity, I will have to assume some basic knowledge of PCI Express. If you're new to this subject, then you can read my [PCIe Primer](#) first.

Altera Cyclone V Hard IP for PCI Express

Altera supplies PCIe IP for many of their family of devices, including Agilex, Arria and Stratix, but we will choose a Cyclone V device as being readily available, low cost and power efficient. Not all variants of the Cyclone V have this Hard IP, but the GX, GT, SX and ST devices have GEN1 and GEN 2 implementations.

This IP can be configured for Root Complex or Endpoint configuration, but we'll stick to an endpoint interface. At the other end, the interface can be either an Avalon streaming (Avalon-ST) or Avalon memory mapped (Avalon-MM) interface, and we need the latter. These are the two main interfaces of the IP, but there's a lot needed to be done to get them configured as required.

For the purposes of this case study, it doesn't really matter which Cyclone V device we use as the IP is the same in all device variants that support it. But we will choose, arbitrarily, a **5CGXFC7D6F31C7** device. We can tell this device has PCIe Hard IP as the character after 5CGX is 'F' instead of a 'B'. We will need this later when we generate the IP simulation model.

EDA Tools

Another reason for choosing the Altera flow (and this applies to Xilinx as well) is that, despite being commercial and licenced EDA packages, there are versions available that are free to download which have everything that's required to generate and use the PCIe hard IP. For many people reading this document they are likely to be using

commercial EDA tools in the work environments, and this case study covers using such an EDA flow, even if using free versions.

The Altera tool chain is the Quartus Prime Tool. There is a licence free SC Lite version that only supports the low-end devices, such as Cyclone, and is missing some advance features such as partial reconfiguration and formal verification, but these restrictions need not concern us. Optionally supplied with Quartus is a free edition of the Siemens Questa logic simulation as the Altera Starter FPGA Edition. This does require a licence, but it is available at no cost, though must be renewed each year. Registering an account is required which can be done via the [FPGA Licensing Support Center](#) and clicking the "sign in" icon in the top bar. The EDA tools can be downloaded from the [FPGA Software Download Center](#), where you can choose between Windows and Linux operating systems. When running the installer, make sure that, under *Questa-Altera FPGA and Starter Editions* that the Starter Edition box is ticked. As a minimum in the Devices section, *Cyclone V device support* must be ticked. To save disk space you may untick any devices you don't want and any add-ons (they can be installed at a later date if required).

Generation of Simulation Model

Before we start on this topic it should be noted that the *pcieVHost* repository has pre-compiled Questa libraries for the Altera PCIe IP simulation model and so the simulation can be run without compiling the Altera source code (the default situation). This step is documented here for completeness to go through the process, and if it is desired to generate and compile the Altera code, removing or moving the `vhd1/testaltpcie/libraries` directory will generate compile the source code when 'make' is next run.

Before we can generate the code for the PCIe EP we need to decide what we want and so a refined specification from our original goals is required. A stated goal was to keep things at a minimum and so we want the simplest valid specification:

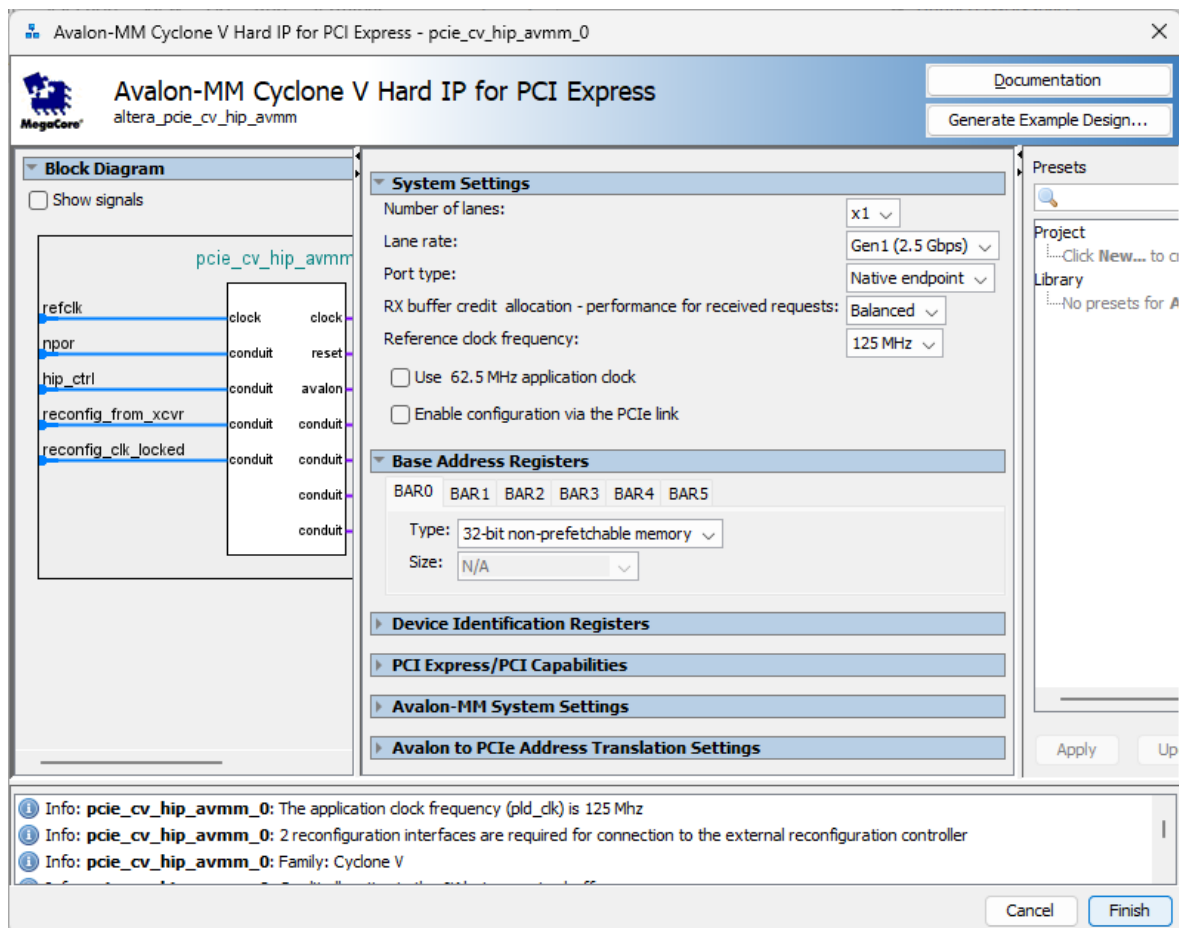
- PCIe GEN 1 specification
- An Endpoint
 - Completer only (no ability to generate requests)
- Single Lane PCIe link
- PIPE interface (no PHY level modelling)
- Single configured 32-bit single Avalon memory mapped master interface
 - 32-bit address
 - 64-bit data

Some things to note here. Altera provide an Avalon-ST or Avalon-MM interface option, and we will use the latter for this exercise so we can do basic reads and writes to address in memory space. The Avalon interface has 32-bit address, but the choice for data width is 64- or 128-bits. We will choose the 64-bit data and do something about this in the logic to have a 32-bit data bus in the design.

Platform Designer and IP Generation

The Quartus tool suite has a graphical sub-program called Platform Designer, which is GUI interface to put together, configure and connect blocks of design for both the supplied IP and for user designs. In this document it would take too long to put together a system completely in Platform Designer, but we will use it to gain access to the PCIe IP and generate the source code. You can gain access to Platform Designer by firing up Quartus and selecting Tools->Platform Designer. You can also fire up Platform Designer from a terminal with the `qsys-edit` command. From Quartus there is also an IP Catalog window, and, under the Interfaces section, there is a PCI Express with "Cyclone Hard IP for PCI Express" as an option. However, I could only get access to the Avalon streaming version of the IP and not the memory mapped version. This isn't the case with Platform Designer, hence using this method.

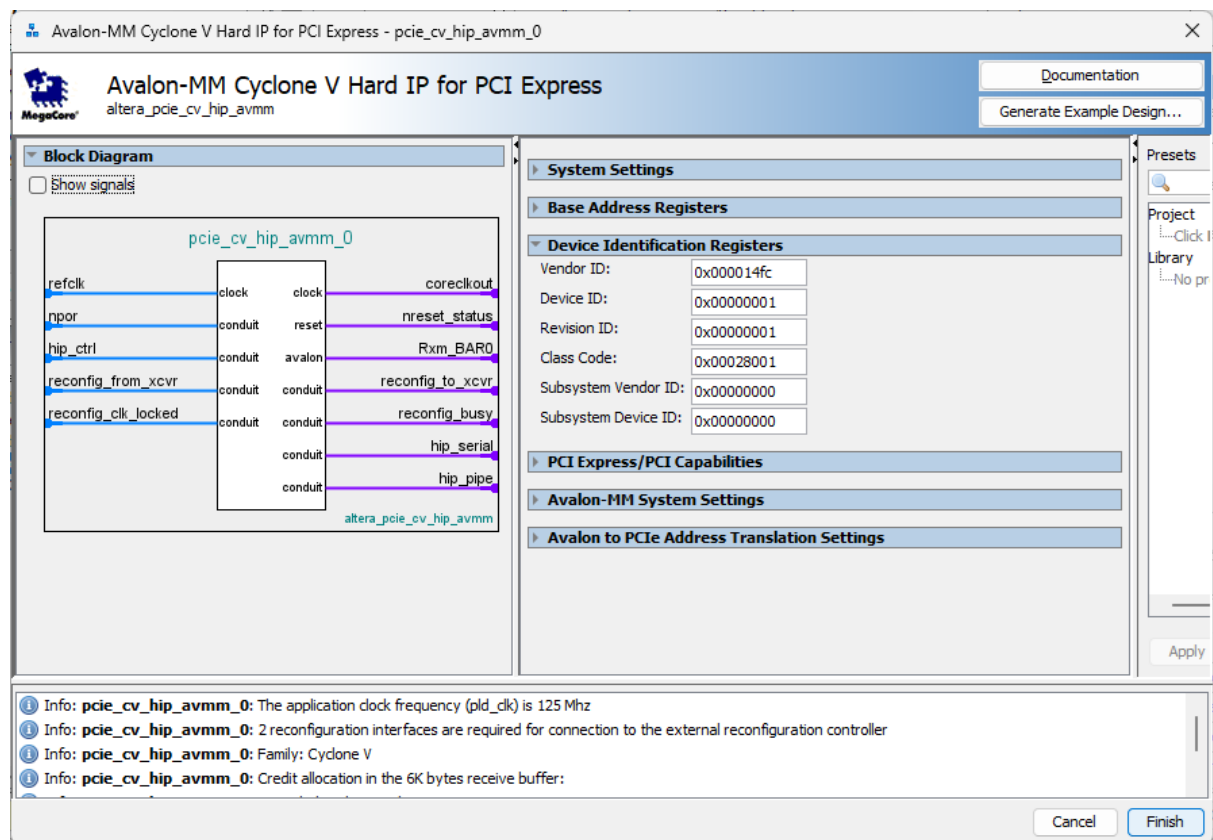
When Platform Designer is running there is a single clock block and an IP Catalog box at the top left. There are various selections under Library, including Interface Protocols which is where a PCI Express tab is found and this has two selections to choose from for the Cyclone V devices. We want the *Avalon-MM Cyclone V Hard IP for PCI Express*, and double clicking this adds the block to the System Contents window and opens up a new configuration window, which we need to configure. Firstly, we want the basic settings for both the PCIe interface and the mappings to the Avalon interface.



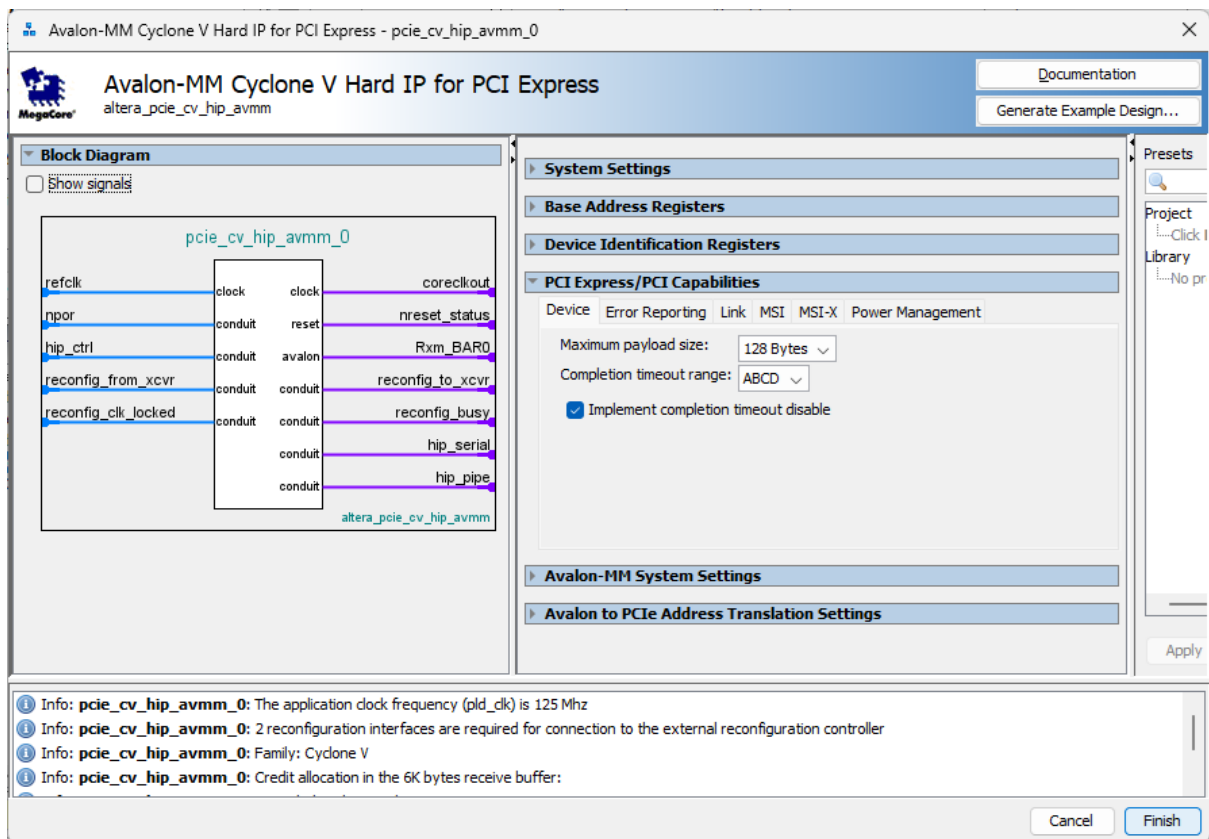
The above diagram shows that a single lane is selected (changed from a default of 4), Gen1 is selected and configured as a native endpoint. It doesn't really matter by RX Buffer credit allocation is changed from Low to Balanced. This is a trade-off between performance and resource usage. The reference clock value defaults to 100MHz but can be selected to be 125MHz. 125MHz is going to be the frequency of the Avalon interface whichever reference clock is being used, so my thinking was that using 125MHz is likely to take less simulation time in generating the interface clock from it. Of course, in a real design there are other factors that may affect this choice. A check box to use a half rate interface (application) clock is available if the user logic needs a slower clock, but we will not need this. We could also select being able to configure the FPGA logic over the PCIe interface if we want, but we will skip this option.

The base address registers (BARs), in the Configuration Space PCI registers, define the sections of relocatable mapped memory blocks a device can have, up to 6. For each BAR we configure there will be a separate Avalon interface. All the BARs are disabled by default, but at least one must be configured and we will configure just BAR0. We only have to choose between 64-bit prefetchable memory and 32-bit non-prefetchable memory, and we configure for the simpler 32-bit option. All the other

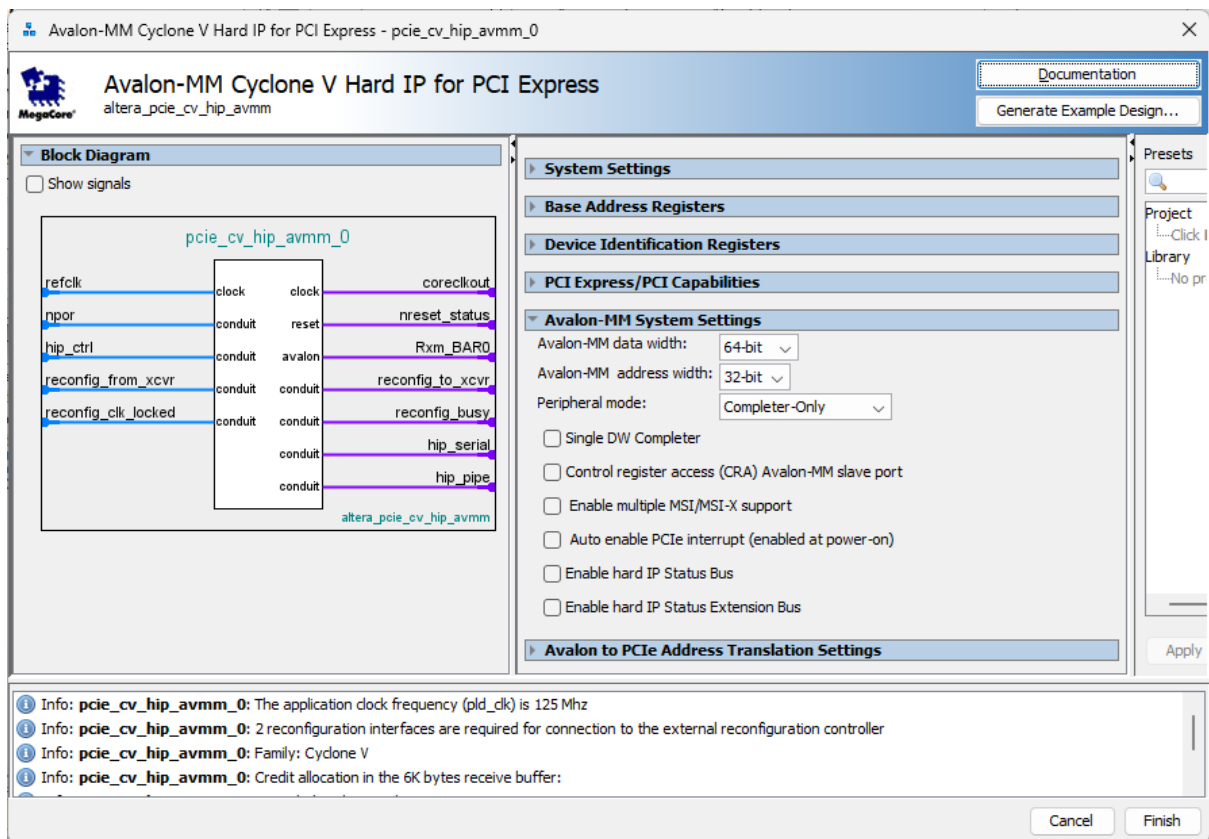
BARs are left at their default Disabled configuration. After the BARs we have the Device Identification Registers.



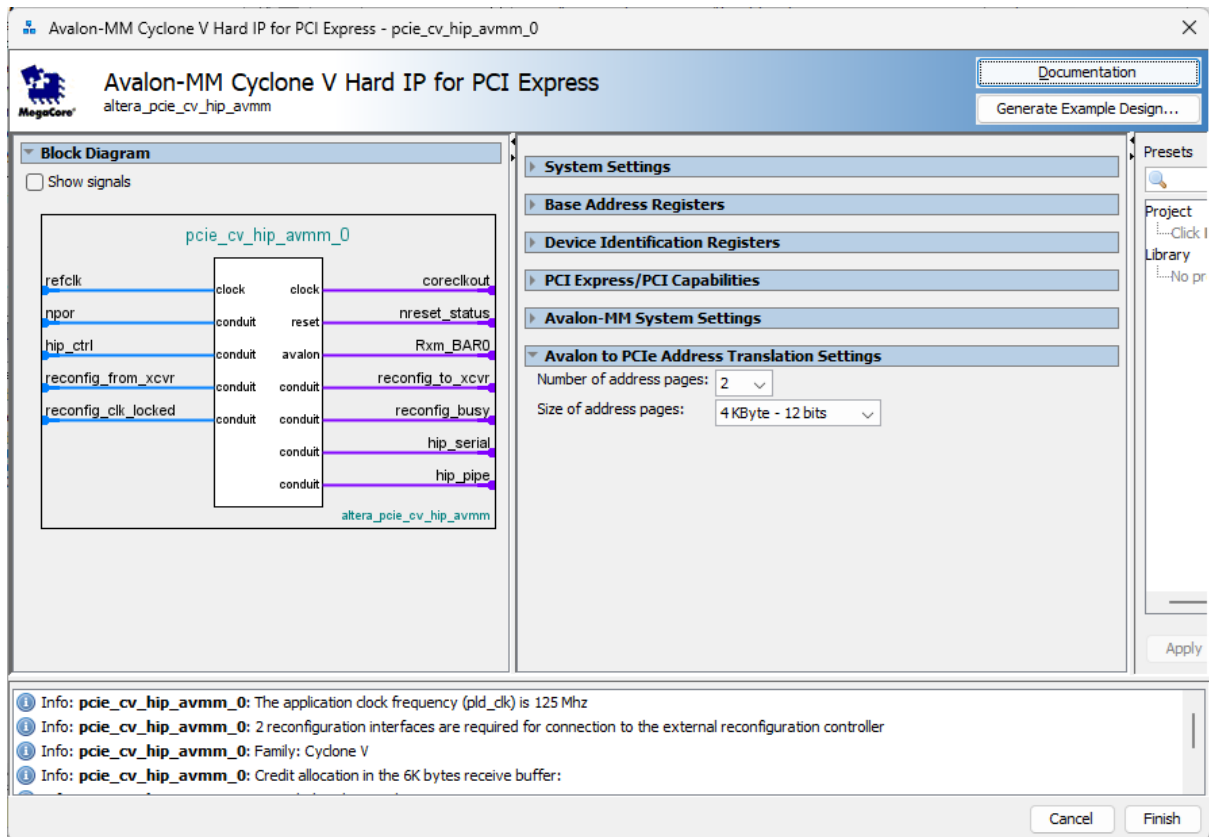
For the test bench that's being constructed, these could actually be left at their default values but, so we can verify that we can read these registers in the tests, the values as shown above have been set, with the Vendor IP set for Quadrics (see [here](#)) and a class code set for a network controller (0x0002) sub-class "other" (0x80) and programming interface "network controller" (0x01). The next section to configure is the PCI Express/PCI capabilities:



This section has multiple tabs, but we only need to change a few from the defaults. The Device Tab shown above uses the default values with the smallest setting for a maximum payload size of 128 bytes. The Completion timeout range is not relevant for devices that don't issue requests and the Implementation completion timeout disable is mandatory for endpoints at GEN2 and higher, so we will leave it even for GEN1. All the other tabs are left at default settings except the Link tab, where the Slot clock configuration check-box is unticked. It doesn't really matter, and this just says it is using its own reference clock and not one supplied from the root complex. The next section configures the Avalon -MM interface:



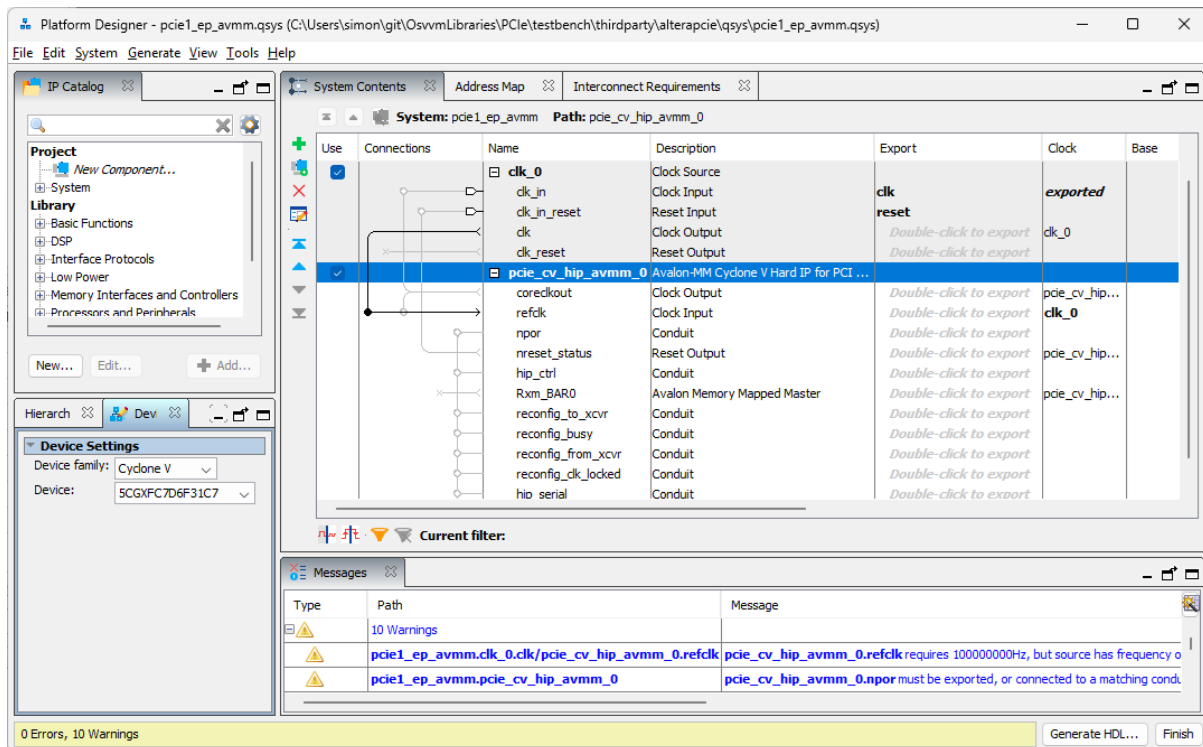
This is largely left to default values, but we change the Peripheral mode to Completer-Only and uncheck the Control Register access port. This is an additional Avalon MM slave port to allow configuration space register updates via memory reads and writes. In the continued goal of minimalist configuration, none of the other optional features are selected. The final section to configure is the Avalon to PCIe Address Translation Settings.



A minimalist approach is taken here as well, with the number of (consecutive) pages left at two, and their size set to the smallest value of 4Kbytes.

Simulation Source Generation

With the configuration done and the Finish button pressed the main System Contents window will show the added PCIe block. There will also be an error saying the block needs a clock. For our purposes I don't know if it's important, but I connected the refclk pin to the clk output of the clock source block. The Platform Designer should now look like the diagram below.



Generating HDL is now just a matter of clicking the Generate HDL... button. This brings up a window to select the type of output (Verilog or VHDL) for synthesis and for simulation, where you want it saved and the name you want to give it (e.g. `pcie1_ep_avmm`).

The simulation model output is what we're interested in, and it doesn't really matter if it's Verilog or VHDL. The top level of the IP will be Verilog anyway, and just a file instantiating the model and a component definition is provided when selecting VHDL. We don't really need this, and I came across issues where the type of the component generics were different from the underlying module, such as a string where the module had an integer for the corresponding parameter. This was also true if the instantiating module was Verilog, but it didn't give compilation issues though did appear to cause issues when simulated. Therefore, I bypassed these optional files and created a wrapper for the Altera IP in SystemVerilog, and it is this that will be instantiated in the VHDL environment.

Wrapping the Altera IP

The top level module for the Altera IP (`altpcie_cv_hip_avmm_hwtc1`) has 257 parameters and 359 ports. That's quite intimidating, to say the least. For this exercise and the configuration that was set up for the IP, most of these parameters can be left at their default settings and most of the ports will be unused. It will be prudent to abstract away the details of the parameter settings and to only expose the signals the

we are going to need in the test bench. The wrapper will be in SystemVerilog and will require a much, much less complicated component definition to instantiate in VHDL than the Altera IP directly. This wrapper module is called pcie1epavmm, and the source code can be found in the vhd1/testaltpcie/svlog/pcie1epavmm.sv file.

Setting of Altera IP Parameters

As mentioned above, many of the parameters can be left at their defaults and so the wrapper does not need to set a value for all the possible 257 parameters. The parameters that are set are shown in the code fragment below:

```
altpcie_cv_hip_avmm_hwtcl # (

    // Define general specification
    .port_type_hwtcl          ("Native endpoint"),
    .gen123_lane_rate_mode_hwtcl ("gen1"),
    .pll_refclk_freq_hwtcl    ("125 MHz"),           // default "100 MHz"
    .lane_mask_hwtcl          ("x1"),                // default "x4"

    // Set fields in configuration space
    .vendor_id_hwtcl          (5372),                // default 4466
    .device_id_hwtcl          (1),                   // default 57345
    .revision_id_hwtcl        (1),                   // default 1
    .class_code_hwtcl         (163841),              // default 16711680
    .subsystem_vendor_id_hwtcl (0),                   // default 4466
    .subsystem_device_id_hwtcl (0),                   // default 57345
    .max_payload_size_hwtcl   (128),                 // default 256
    .bar0_io_space_hwtcl      ("Disabled"),
    .bar0_64bit_mem_space_hwtcl ("Disabled"),
    .bar0_prefetchable_hwtcl  ("Disabled"),
    .bar0_size_mask_hwtcl     (12),                  // default "256 MBytes - 28 bits"

    // Define flow control parameters
    .vc0_rx_flow_ctrl_posted_header_hwtcl (16),       // default 50
    .vc0_rx_flow_ctrl_posted_data_hwtcl   (16),       // default 360
    .vc0_rx_flow_ctrl_nonposted_header_hwtcl (16),    // default 54
    .vc0_rx_flow_ctrl_nonposted_data_hwtcl (0),
    .vc0_rx_flow_ctrl_compl_header_hwtcl   (0),        // default 112
    .vc0_rx_flow_ctrl_compl_data_hwtcl     (0),        // default 448
    .cpl_spc_header_hwtcl                   (67),       // default 195
    .cpl_spc_data_hwtcl                     (269),      // default 781

    // A single receive detector for the 1 lane
    .single_rx_detect_hwtcl                 (1),        // default 0

    // We don't need a configuration space Avalon access port
    .CG_IMPL_CRA_AV_SLAVE_PORT              (0),        // default 1

    // Some required parameters for Avalon to PCIe bridge settings
    .CB_A2P_ADDR_MAP_NUM_ENTRIES            (2),        // default 1
    .a2p_pass_thru_bits                     (12),       // default 24

) altcie_i
(
    // **** PORT connections **** //
);
```

In the above code fragment, the parameters have been grouped by function and the comments on the right hand side give the default values where the settings are

changed. There are some settings that remain at default values but are included for completeness and documentation for the group they belong to. Note the `bar0_size_mask_hwtc1` setting where the default value is a string, but it is set to an integer to give the number of bits. Using a string in the code failed to set the correct configuration and I had to reverse engineer some code to find what was required. This is an example of where the auto-generated code did not entirely match the required parameter type.

Most of these parameters should be familiar with the settings made in Platform Designer. Only BAR0 parameters are set and using mostly the default values, except for the aforementioned mask setting to give 4Kbytes.

The VC0 parameters weren't set directly in Platform Designer, but by the minimum/low/balanced selection, though these can be changed here to suit specific requirements.

The `single_rx_detect` parameter specifies the number of receiver detect blocks required and should match the number of lanes. Leaving this at the default of 0 or specifying more than the active lanes seems to cause issues. The last two parameters simply reflect the address translation settings from the Platform Designer configuration. All the other 230 parameters remain at their default settings and thus the configuration has been narrowed down to just a handful of required changes. A similar exercise can be done for the ports.

Extraction of Required Signals

Of the 359 ports of the Altera IP module, most are duplicates for each of the possible different lanes in the PCIe link or for Avalon interfaces for each of the six BARs. Since we have one lane and one Avalon interface, we can leave most of these unconnected—with a few exceptions which I'll return to. Apart from the two main interfaces, there are just a few additional ports for clocks and reset, and some simulation model control and status outputs. The code fragment below shows the port connections used:

```

altpcie_cv_hip_avmm_hwtcl # (

// **** Parameters **** //

) altpcie_i (

// Reset signals
.pin_perst      (nReset),           // input
.npor           (nReset),           // input

// Clocks
.refclk         (RefClk),           // input
.coreclkout     (coreclkout),       // output

// PIPE signals for simulation only
.simu_mode_pipe (1'b1),             // input
.sim_pipe_pclk_in (PipeClk),        // input
.sim_pipe_rate   (Rate),            // output [1:0]
.sim_ltssmstate  (LtssmState),      // output [4:0]

// Input Pipe interface
.phystatus0      (PhyStatusInt),     // input
.rxelecidle0     (RxElecIdleInt),    // input
.rxstatus0       (RxStatusInt),      // input [2:0]
.rxdata0         (RxData),           // input [7:0]
.rxdatak0        (RxDataK),          // input
.rxvalid0        (~RxElecIdleInt),   // input

// Output Pipe interface
.eidleinfersel0  (EidleInferSel),    // output [2:0]
.powerdown0      (PowerDown),        // output [1:0]
.rxpolarity0     (RxPolarity),       // output
.txcompl0        (TxCompliance),     // output
.txdetectrx0     (TxDetectRx),       // output
.txlecidle0      (TxElecIdle),       // output
.txmargin0       (TxMargin),         // output [2:0]
.txdeemph0       (TxDemph),          // output
.txswing0        (TxSwing),          // output
.txdata0         (TxData),           // output [7:0]
.txdatak0        (TxDataK),          // output

// Avalon Rx Master BAR0 interface
.RxmWrite_0_o    (Bar0Write),        // output
.RxmAddress_0_o  (Bar0AddressAligned), // output [AVALON_ADDR_WIDTH-1:0]
.RxmWriteData_0_o (Bar0WriteDataWide), // output [avmm_width_hwtcl-1:0]
.RxmByteEnable_0_o (Bar0ByteEnableWide), // output [(avmm_width_hwtcl/8)-1:0]
.RxmBurstCount_0_o (Bar0BurstCount), // output [avmm_burst_width_hwtcl-1:0]
.RxmWaitRequest_0_i (Bar0WaitRequest), // input
.RxmRead_0_o     (Bar0Read),         // output
.RxmReadData_0_i (Bar0ReadDataWide), // input [avmm_width_hwtcl-1:0]
.RxmReadDataValid_0_i (Bar0ReadDataValid), // input

);

```

The reset inputs are almost the same function, but the npor input can also have any local reset signalling combined with the signal into pin_perst. For this exercise, we can use the same reset signal for both. For the clock there is the reference clock input, which is configured for 125MHz operation in this case, and a coreclkout clock that the application logic must use for connection to the Avalon bus, derived from the reference clock and also 125MHz.

The next block of signals is for simulation only and are not present for synthesis. They are provided by Altera to speed up simulation by not modelling the PHY layer. As the objective here is not to test the Altera PCIe implementation, but to set up a test environment that connects to it, we can take advantage of this facility. By setting the `simu_pipe_mode` input to `1'b1`, the interface will be the standardised PIPE interface (PHY Interface for PCI Express) between MAC and PHY logic. The *pcieVHost* model supports the PIPE interface data signals, but also supports 8b10b encoded output, and has serialiser wrappers for bitstreams on each lane. So it is possible to use the model with the IP configured in full serial mode. If using the PIPE mode, a PIPE clock input is required and, for GEN1, this is 250MHz. A couple of useful outputs are also provided to give the current rate the IP is in (GEN1, GEN2 etc.) and the LTSSM state. For the LTSSM, Questa can be configured to have a radix to decode this output with the following added to a `.do` file as part of the waveform setup to display signals and a `wave.do` file can be found in the `vhd1/testaltltpcie` directory with these settings.

```
radix define LtssmState {
    "5'b00000" "Detect.Quiet",
    "5'b00001" "Detect.Active",
    "5'b00010" "Polling.Quiet",
    "5'b00011" "Polling.Compliance",
    "5'b00100" "Polling.Configuration",
    "5'b00101" "Polling.Speed",
    "5'b00110" "Config.LinkwidthStart",
    "5'b00111" "Config.LinkAccept",
    "5'b01000" "Config.LaneNumAccept",
    "5'b01001" "Config.LaneNumWait",
    "5'b01010" "Config.Complete",
    "5'b01011" "Config.Idle",
    "5'b01100" "Recovery.RcvLock",
    "5'b01101" "Recovery.RcvConfig",
    "5'b01110" "Recovery.Idle",
    "5'b01111" "L0",
    "5'b10000" "Disable",
    "5'b10001" "Loopback.Entry",
    "5'b10010" "Loopback.Active",
    "5'b10011" "Loopback.Exit",
    "5'b10100" "Hot.Reset",
    "5'b10101" "L0s",
    "5'b11001" "L2.Transmit.Wake",
    "5'b11010" "Speed.Recovery",
    "5'bzzzzzzzzzz" "UNCONNECTED",
    "5'bxxxxxxxxxx" "ERROR",
    -default hexadecimal
}
```

The next set of signals are the inputs to the PIPE interface. The `phystatus0`, `rxlecidle0` and `rxstatus0` signals are involved in resetting the PHY (more shortly), and these are followed by the received 8-bit data and K bits and a valid input, set when the receiver is stable and receiving code inputs for the lane.

The PIPE output signals give a set of status signals which I've routed to the wrapper modules ports. I won't go through each of these as they are documented in Part 1 of

my [PCIe Primer](#). The outputs also include the 8-bit transmit data and K signals for sending upwards on the PCIe link.

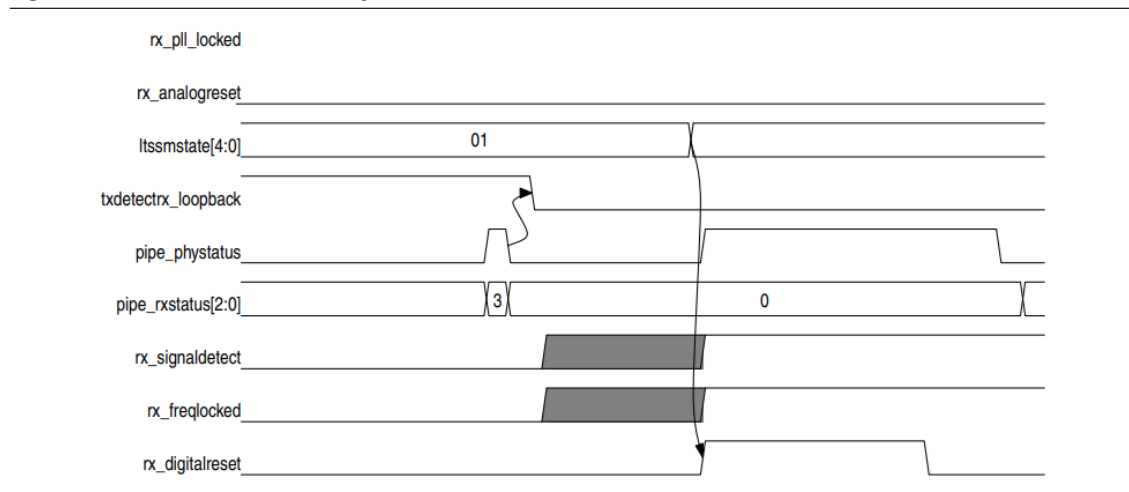
The final signals are for the Avalon Memory Mapped master interface associated with BAR0. Since the data is 64-bits wide the wrapper contains some simple logic to map these to 32-bit wide data and (for writes) 4-bit byte enables. This is designed to allow single data transactions of a word, half-word or byte only. More sophisticated logic is needed for larger transactions, or the wider interface can be connected directly to the application logic. For the minimalist ethos taken here, it's envisaged that this BAR interface is an example for reading and writing 32-bit registers. If a data channel is needed, then a new BAR can be configured for larger transfers, in which case a wide data bus is probably needed to keep logic clock frequencies down.

In addition to the above-mentioned ports, some of the inputs to the unused Avalon bus ports for the other BARs had to be tied off to 0, to avoid X propagation to the used BAR0 Avalon interface port. In particular, the `RxmWaitRequest_<num>_i` and `RxmReadDataValid_<num>_i` inputs. This was not true for the unused PIPE inputs.

PHY Reset Sequencer

In the [Cyclone V Hard IP for PCI Express User Guide](#), is detailed the RX transceiver reset sequence required and the timing diagram from this document is shown below.

Figure 9-3. RX Transceiver Reset Sequence



The wrapper logic contains a simple open-loop sequencer to perform this reset signalling. The sequencer drives the `phystatus0`, `rxlecidle0` and `rxstatus0` signals as shown, with the key settings to assert `phystatus0` for a cycle with `rxstatus0` at 3, then de-assert with `rxstatus` at 0. The sequencer drives PIPE inputs directly to the Altera IP module, disconnecting these from the wrapper's ports. However, a parameter

(DISABLE_INT_RESET_SEQ) is provide to disable this internal sequencer and allow the PIPE signals to be driven externally.

Compiling the IP Source

As mentioned earlier, the test bench supplies pre-compiled libraries for the Altera PCIe model, but the test bench also supplies a pre-configured Platform Designer .qsys file from which the Altera PCIe IP can be generated. As part of the output from this it also supplies a TCL script to compile this source. As well as firing up the Platform Designer GUI and hitting the "Generate HDL..." button, this can also be done from a terminal command, qsys-generate, with a just a few arguments. For example, from the vhd1/testaltpcie directory:

```
qsys-generate --simulation=VERILOG          \  
              --output-directory=pcie1_ep_avmm \  
              ./qsys/pcie1_ep_avmm.qsys
```

This will generate a set of files in a folder pcie1_rp_avmm. Under this directory is a sub-directory called simulation. The source code for the model is in the submodules directory, and there is another sub-directory called mentor that contains a TCL script to compile the source code—msim_setup.tcl. To use this script to compile the code then, again from the vhd1/testaltpcie directory, the following command can be used:

```
vsim -c -do "set QSYS_SIMDIR "[pwd]/pcie1_ep_avmm/simulation;          \  
            source pcie1_ep_avmm/simulation/mentor/msim_setup.tcl; \  
            dev_com;                                                    \  
            com;                                                         \  
            quit"
```

After this command is run, a libraries directory will be created/updated containing the pcie_cv_hip_avmm0 library that has the compiled code. When running the simulation, we can map this library location to a logical name using vmap:

```
vmap pcie_cv_hip_avmm_0 libraries/pcie_cv_hip_avmm_0
```

This mapping should be done whether compiling from scratch or using the pre-compiled libraries. The logical name can then be used, when compiling the wrapper source code and running the simulation during elaboration, to find the Altera PCIe model library. The wrapper SystemVerilog, then, is compiled as:

```
vlog -sv -l pcie_cv_hip_avmm_0 svlog/pcie1epavvm.sv -work work
```

From this point we can compile all the other HDL source for the test bench and compile the C and C++ for the *pcieVHost* and *VProc*. This is all conveniently gathered up into a `makefile` for compiling with `make`, including the steps above to compile the Altera PCIe library from scratch or using the pre-compiled libraries.

Conclusions

In this first of a part of the document, to start, a PCIe IP implementation has been chosen after selection of an FPGA vendor, a target FPGA device, and the 'flavour' of PCIe interface on offer for that device. From this point, a step-by-step process was given for using the Vendor's EDA tools to configure the PCIe block to the stated requirements and the source code generated suitable for simulation.

The top level IP block was found to have a large parameter count and an even larger port count, but with most of these unchanged from defaults or able to be left unconnected. So, a wrapper module was constructed to instantiate the IP configuring the parameters set only for what is required, and only routing signals to external ports that were required. A sequencer process was also added to perform receiver reset after power on. This abstraction made instantiating the PCIe block much simpler, especially as this was to be instantiated in VHDL as a mixed language component.

Once this wrapper logic was constructed, the steps were explored to use the provided TCL script to compile the Altera PCIe IP for the Siemens Questa logic simulator bundled with the Altera Quartus tools. The next step was given for generating a logical library mapping from the resultant compiled library, and the final step of compiling the wrapper SystemVerilog source code.

At this point, then, we have everything we need to use this IP in a *pcieVHost* based logic simulation environment. This is the subject of the next part, where a test bench is constructed around the *pcieVHost* VIP, along with *mem_model* as a stand in for application logic to be developed using the Altera PCIe IP.

Part 2: The *pcieVHost* test bench

Introduction

In Part 1 of this two part document, we concentrated on setting the purpose on goals of the case study, choosing an IP vendor, followed by a step-by-step walk through the process of configuring and generating the PCIe IP simulation model, then wrapping it in a module to abstract away the details of the model's instantiation to expose only the relevant signals that are required. Details were then given for compiling the PCIe model and wrapper in the target logic simulator.

We can now take this PCIe IP simulation model and construct a logic simulation verification environment using the *pcieVHost* model to drive the third part IP, and also the *mem_model* co-simulation component to stand in place of where the user application logic being developed would be. This, then, serves as a template for such a real-world verification environment, where the memory model can be removed and user logic added and developed.

In this second part, the steps of constructing the test bench's HDL are discussed, along with some introductions to the two VIP components being employed, and the details of the *pcieVHost* test program using the *pcieVHost* features to bring the IP up from power-on-reset, through PHY receiver reset, the LTSSM states From Detect.Quiet to L0, DLL VC0 flow control initialisation, and then configuration space exploration and enumeration. At this point we can talk to the user application logic (*mem_model*) and do memory reads and writes.

So, let's jump in at the deep end and discuss the overall VHDL testbench architecture that we want to construct.

The HDL Test Bench Architecture

We now have a SystemVerilog module from part 1 that can be instantiated within a VHDL environment, but we must construct a VHDL component first. This step can, obviously, be skipped in a Verilog/SystemVerilog environment. The full component definition can be found in the `vhd1/testaltpcie/test_pkg.vhd` file in the *pcieVHost* github repository. In order to bind to the SystemVerilog module correctly, the names (and cases) of the ports, generics and component names must match the ports, parameters and module names. No fancy types have been used, so the ports can all be of `std_logic` or `std_logic_vector` as appropriate. We can now instantiate the wrapper with a component reference:

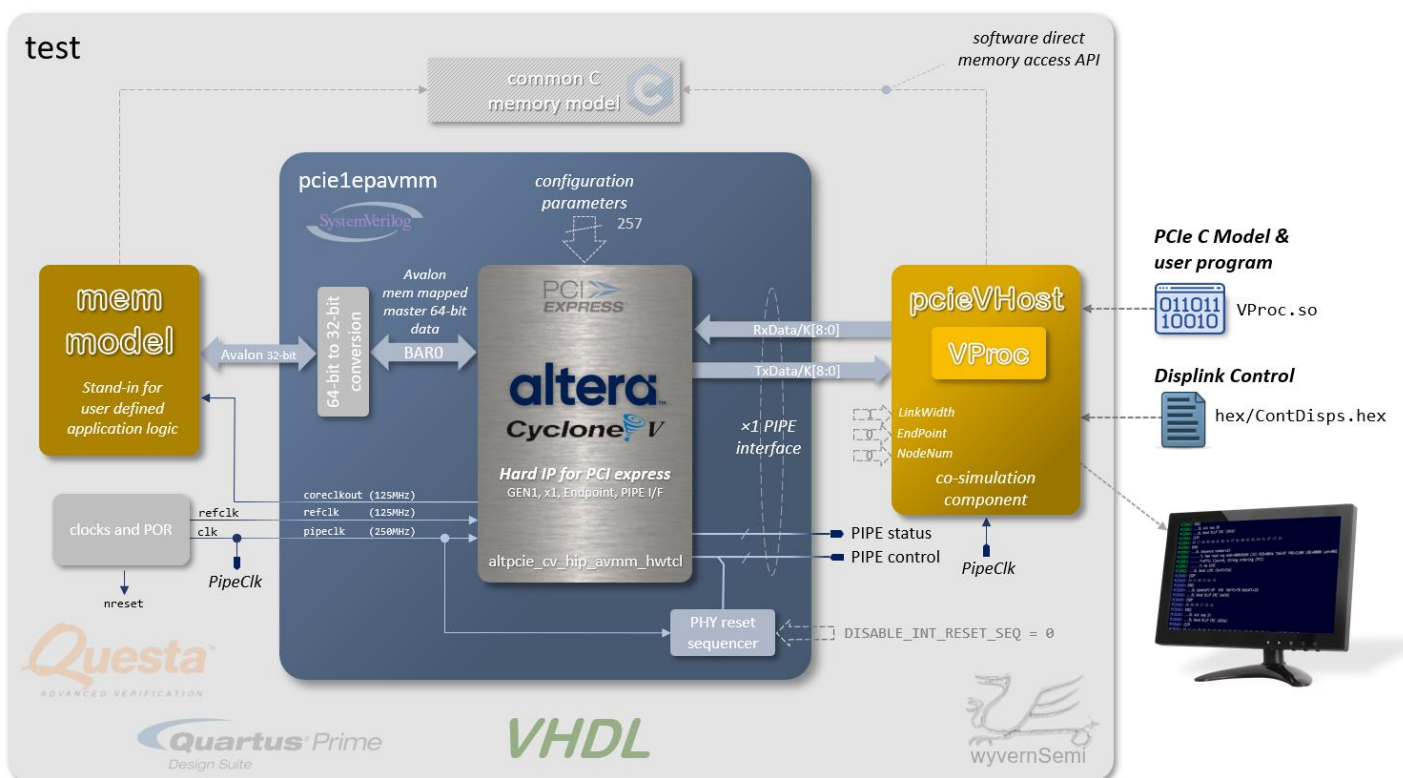
```

use work.test_pkg.all;

ep_i : component pcie1epavmm
generic map (
  --- Generics ---
)
port map (
  --- ports ---
);

```

We now have something with which to start constructing our VHDL test bench around. The following diagram gives the architecture of the test bench to be used with the Altera PCIe IP.



At the heart of the test bench is the **pcie1epavmm** component, and its internal structure is also shown. Connected to the PIPE TX and RX Data and K signals is a **pcieVHost** component, configured as a root complex, with a link width of 1 lane. The rest of the PIPE signals for this test bench example provide status, but the only inputs required are driven by the sequencer in the **pcie1epavmm** module.

Connected to the Avalon bus of the Altera IP wrapper is the **mem_model** IP. This is a stand in for the user application logic which would normally be connected to the memory mapped bus, but it allows us to write from the **pcieVHost**, through the PCIe interface to this memory, and then read back these memory locations to verify.

The test bench has some functionality to generate a couple of clocks and an active low reset. A 125MHz reference clock and a 250MHz pipe clock are generated, both routed into the Altera PCIe wrapper, and with the *pcieVHost* component clocked with the pipe clock. The memory model is clocked with the 125MHz output from the Altera IP.

We now have an HDL test bench ready to use that can serve as a template for developing and testing application logic with the *Cyclone V Hard IP for PCI Express*. There is still a lot that needs to happen before we can communicate with the application over the PCIe link, and this is where the *pcieVHost* model comes into its own.

The *pcieVHost* and *mem_model* VIP

Having mentioned that we will be using the *pcieVHost* and *mem_model* verification IP it might be worth summarising what these do if this is new to you. If you've read some of my other articles on this subject, or are using these components already, then you can skip this section.

Both of these are co-simulation components with *pcieVHost* based around the *VProc* virtual processor IP and *mem_model* using more traditional methods to call C or C++ code from HDL. *VProc* allows a user to write C or C++ code and natively compile it for the host PC or workstation and run this code on an HDL component with a memory mapped bus master instantiated in a logic simulation. An API is provided to instigate read and write transactions on the HDL bus from within the user program. I've written about this in more detail [here](#). Beyond these bus transactions, *VProc* can be used to generate any arbitrary protocol with a C or C++ model and *pcieVHost* is just such a model, written in C and using *VProc* to drive PCIe link signals in an HDL logic simulation. I've written about the PCIe C model in detail [here](#), and about how *VProc* is used for arbitrary protocols [here](#), using the PCIe C model as an example. Below are listed some of the key features of the *pcieVHost* model.

- All lane widths up to $\times 16$
- Internal memory space model accessed with incoming write/read requests (can be disabled)
- Auto-generation of read completions (can be disabled)
- Auto-generation of 'unsupported' completions (can be disabled)
- Auto-generation of Acks/Naks (can be disabled)
- Auto-generation of Flow control (can be disabled)
- Auto-generation of Skip OS (can be disabled)
- User generation of all TLP types

- Memory Reads/Writes
 - Read completions
 - Config Reads/Writes
 - IO Reads/Writes
 - Messages
- User generation of all DLLP types
 - Acks/Naks
 - Flow control
 - Power management
 - Vendor
- User generation of all training sequences
- User generation of all ordered sets
- User generation of idle symbols
- Proper throttling on received flow control
- Lane reversal and inversion
- Serial input/output wrapper HDL
- Programmable FC delay (via Rx packet consumption rates)
- Programmable Ack/Nak delay
- Programmable Completion delay
- PIPE interface mode support
- Disabling of Scrambling
- LTSSM (partial implementation)
- Built-in PCIe link formatted display output

The PCIe C model provides its own API above that provided by *VProc*, and details of this can be found in the [pcieVHost documentation](#). We will look at some of this later, though.

The memory model doesn't run a program, but uses normal logic programming interface calls (VPI, FLI, DPI etc.) to read and write a full 64-bit memory model in C. This was an offshoot of the internal memory model of *pcieVHost* to be a stand-alone memory model, when not using the PCIe model. However, when the *pcieVHost* model is present, it can access and use its internal memory model code in place of a separate memory space. This has the advantage that user code running on the *pcieVHost* model can access memory directly via an API, and these locations will be the same when accessed via the *mem_model*'s HDL ports. So, data can, for example, be pre-loaded in memory at high speed, without consuming simulation cycles, and then accessed by the HDL logic in the simulation.

By default, the memory model HDL has a set Avalon compatible interfaces, one a memory mapped bus slave, and a couple as TX and RX burst interfaces. Wrappers are provided to support for APB, AHB and AXI protocols as well. I have written more details of the memory model as part of an article on PLIs and co-simulation, [here](#) (see Part 1: Real-world Use Case). The API for the model can be found in the [mem_model documentation](#).

The *pcieVHost* Test Program

In order to drive the PCIe link from *pcieVHost* we need a C or C++ test program to run on it. An API is provided to generate the PHY, DLL and transaction patterns and this is given in full detail in the [pcieVHost documentation](#). However, a brief look at some relevant API methods is pertinent to this discussion in the context of the test program being to be used. For this purpose we will use the C++ API class `pcieModelClass`.

Since the PCIe C model is running on VProc, we need a VProc main entry point that matches its "node" number. From the test bench diagram above this node number is set at 0, so we must have a function, with C linkage, called `VUserMain0`. The code fragment below gives the basics of the test program to be run. It's only a few lines but actually it does a lot of heavy lifting before we get to the business of sending test transactions.

```
#include <stdio.h>
#include <stdlib.h>

#include "pcieModelClass.h"

extern "C" void VUserMain0(int node)
{
    // Create an API object for this node
    pcieModelClass* pcie = new pcieModelClass(node);

    // Initialise PCIe VHost, with input callback function and no user pointer.
    pcie->initialisePcie(VUserInput_0, NULL);

    // Configure Model for PIPE mode
    pcie->configurePcie(CONFIG_DISABLE_8B10B);

    // Do Link training and initialise the link for 1 lane
    pcie->initLink(1);

    // Initialise DLL flow control
    pcie->initFc();

    // **** TEST CODE HERE **** //

    // Halt the simulation
    VWrite(PVH_FINISH, 0, 0, node);
}
```

The first line in `VUserMain0` creates the API object for the PCIe C model, and binds this to node 0, with the argument passed in. The model then needs initialising with an `initialisePcie` method which takes two arguments. The first is a pointer to a function to use as a callback function, invoked each time the model receives a packet that it didn't handle itself. For this test, this will be to handle the completions of non-posted transactions. The second argument is a pointer that will be supplied to the callback function from user code when it's invoked. In this case we won't use this facility. We'll return to the callback a little later.

The *pcieVHost's* C model is highly configurable, but for this test the most relevant configuration is to put the model into PIPE mode, which is done on the next line by disabling 8b10b encoding and decoding. This puts the model in a mode where the unencoded codewords are placed on the 10 bit output, but only using the bottom 9 bits, with bits 7 down to 0 being the data, and the K bit (to indicate a control symbol) in bit 9. By default, scrambling is still enabled, but this could be disabled as well if required. At this point we can start sending out and receiving codes on the link.

Whilst the API provides means to send out ordered sets and training sequences, it also provides a (partial) implementation of the LTSSM that uses these API methods to go from a powered down (`Detect.Quiet`) state and through the training sequence to the L0 powered up state. This is instigated with a call to the `initLink` method that takes a single argument specifying the number of lanes—in this case 1. This method will not return until it reaches L0. A similar high level method is also provided to initialise the flow control for Virtual Channel 0 (VC0), with the `InitFc` method. Again, this sits on top of the DLL API calls to send and received `InitFc` DLL transactions. At this point we are in a position to send transaction layer packets (TLPs) and can start testing proper.

Received Completion Callback Function

Before discussing the main test code, though, we must circle back to the callback function mentioned earlier. This will be called every time an unhandled packet is received by the model. For the configuration we have set, this should only be completions for any non-posted TLP request that the test program sent. In this test code, making the returned data available to the main code uses a very simple mechanism, and the basic structure of the callback is shown in the code fragment below:

```

static uint8_t      ipbuf[MAX_PAYLOAD_BYTES];
static int          ip_byte_count = 0;

static void VUserInput_0 (pPkt_t pkt, int status, void* usrptr)
{
    // If a completion with data, extract the TPL payload data and display
    if (pkt->ByteCount)
    {
        // Get a pointer to the start of the payload data
        pPktData_t payload = GET_TLP_PAYLOAD_PTR(pkt->data);

        for (int idx = 0; idx < pkt->ByteCount; idx++)
        {
            ipbuf[idx] = payload[idx];
        }
        ip_byte_count = pkt->ByteCount;
    }

    // Once packet is finished with, the allocated space *must* be freed.
    // All input packets have their own memory space to avoid overwrites
    // with shared buffers.
    DISCARD_PACKET(pkt);
}

```

The callback is sent the entire received packet in the pkt pointer argument along with any user pointer (not used here). A completion can be received with no data payload but if it has data, then the ByteCount will be non-zero. If data is available, a pointer is extracted to the start of the data within the packet and then this data is copied to a static buffer. After this, the byte count is saved off to a static variable, and this is used as the crude communication method between this callback and the main test program to indicate data is available. As we'll see later, the main program can wait for data to arrive by polling the ip_byte_count variable until non-zero, process the data and then clear the count. In the polling loop the code must also send out idle symbols with the sendIdle method. In the actual test code this is wrapped up in a provided WaitForInput macro.

Thread Safety

A slight technical aside here. The main user program, or any sub-programs called from it, are running in a thread, separate from the thread(s) that the main simulator functionality is running in. However, the callback is called from within a simulator thread (via the VPI) and is thus, technically, asynchronous to the main program—but we're attempting to communicate between these threads with a simple integer variable and a buffer. Is this safe? Actually, yes. The way VProc works and the blocking nature of VPI calls, ensures that the simulator thread(s) and VProc node threads (of which there can be multiple) can only have one active at any given time. Thus, if we are in the callback code, setting the byte count variable, or in the main user program, reading the variable, the other must be inactive and stalled. This is why we can have such a simple arrangement as here though, of course, more sophisticated systems for

handling, say, out-of-order completions, can be constructed and the API does include `waitForCompletion` methods. My article on [The VProc Virtual Processor Verification IP](#) explains threading with thread safety and how this all works in greater detail.

TLP Test Code

From the original stated goals, we need to do a basic configuration exploration and enumeration and then some memory writes and reads and verify the data integrity. For this we will need the API methods listed below:

```
pPktData_t cfgWrite (const uint64_t offset,
                    const PktData_t *data,
                    const int length,
                    const int tag,
                    const uint32_t rid,
                    const bool queue = false,
                    const bool digest = false);

pPktData_t cfgRead (const uint64_t offset,
                   const int length,
                   const int tag,
                   const uint32_t rid,
                   const bool queue = false,
                   const bool digest = false);

pPktData_t memWrite (const uint64_t addr,
                    const PktData_t *data,
                    const int length,
                    const int tag,
                    const uint32_t rid,
                    const bool queue = false,
                    const bool digest = false);

pPktData_t memRead (const uint64_t addr,
                   const int length,
                   const int tag,
                   const uint32_t rid,
                   const bool queue = false,
                   const bool digest = false,
                   const bool lock = false);
```

These methods all follow a very similar format with a byte offset or byte address, a pointer to a data buffer for the write methods, a payload length, a tag (incrementing each TLP sent, modulo 256), a requestor ID (anything valid, such as the node number) and a couple of optional parameters to queue the request rather than send, and to add an ECRC digest. The memory read method also has a parameter to set if a locked read or not. The *pcieVHost* model has many more methods for generating TLPs and other packets, but this is all that is required for our purposes.

To generate a write transaction the appropriate method is called. If the `queue` argument is `false` then the call is blocking and the transaction already sent when the method returns. For the read methods, when not queued, the request is sent on return, but the data is not available. So, after the read method calls, the `waitForInput` macro is called

to stall (sending idle symbols) until the data is available, which can then be extracted from the `ipbuf` and processed as required. Note that, until the next API call, the simulation is stalled, so no new data will arrive to stomp over the input buffer. If the data may be required later, then it should be saved off to a buffer of the main test program. In the test program for this exercise, the data is used immediately, so this is not required.

The test code, using these methods will, then, do the following steps:

- Does configurations space reads, and prints out contents, from
 - PCI space Vendor ID and Device ID
 - PCI space Command and Status registers
 - PCI space Revision ID and Class Code
 - PCI space Header Type
 - PCI space BARs 0 to 5
- Does a walk through the capabilities structures
 - Prints out found capabilities type and contents
- Configures BAR0 for address `0x00010000` with a configuration space write
- Enables memory accesses in Command register with a configuration space write
- Writes a 32-bit word to memory with a memory write command
- Writes a 16-bit word to memory with a memory write command
- Reads back first word with a memory read command and verifies
- Reads back second half-word with a memory read command and verifies
- Waits for some cycles
- Finishes the simulation

The last two steps use the `sendIdle` to wait some cycles, and then a low level *VProc* function call (`VWrite`) to write to a predefined “virtual” address to finish the simulation. The test code discussed here is, necessarily, an abbreviated version, but the full code can be found in `vhdl/testaltpcie/usercode/VUserMain0.cpp`.

Building and Running the Test

We now have all the components we need to run the test—we have test bench HDL instantiating a wrapped Altera PCIe IP simulation model with some logic for resetting the receiver, a memory model to act as a place holder for application logic, the *pcieVHost* to drive the PCIe link, a test program to run on the *pcieVHosts*’s *VProc* and some test logic to generate clocks and reset. All that’s left is to build and run the simulation.

Compiling and Running with the Make File

A makefile is provided to hide away the details of the build and run process and make using the test bench easier. It uses the `files_vhdl.tcl` script to compile the VHDL testbench files and will compile the user C++ test code, along with *VProc* and *pcieVHost* C/C++ code, to produce a *VProc.so* shared object library file that the simulator will load at run time. Various build and run options are provided and executing the command `make help` produces the following output:

```
make help      Display this message
make           Build C/C++ code without running simulation
make sim       Build and run command line interactive (sim not started)
make run       Build and run batch simulation
make rungui/gui Build and run GUI simulation
make clean     Clean previous build artefacts
make cleanlib  Clean previous pcie_cv_hip_avmm_0 library build artefacts
```

The simplest command to build and run everything is a `make run` command. This will compile everything, including the Altera PCIe library if `libraries` is not present, and run a batch simulation. A GUI simulation can be run with `make gui`, and a `wave.do` file is provided to set up some waves in the waveform display window.

Two clean options are given to clean up everything except the Altera PCIe library (`clean`) and to clean the Altera PCIe library (`cleanlib`). This latter target cleans out the compiled Altera IP library.

Configuring the *pcieVHost* Link Display Output

One last input to the simulation we need to discuss is controlling the link formatted display output. *pcieVHost* has the ability to decode the data on the link and display as formatted output to stdout on the console that the simulation was run from. A file is read at run-time from `hex/ContDisps.hex`. This is a text file, and the provided file has the following contents:

```
// Example ContDisps.hex file
// Copy to the appropriate test/hex directory, and edit as necessary.

//           +8           +4           +2           +1
// ,---> 11 - 8:   DispNoColour   DispEnIfEp   DispEnIfRc   DispTx
// |,-->  7 - 4:   DispRawSym     DispPL       DispDL       DispTL
// ||,-->  3 - 0:   Unused        DispStop     DispFinish   DispAll
// ||| ,-> Time (clock cycles, decimal)
// ||| |
000 0000000000000
370 000000005892
002 009999999999
```


This file has a list of number pairs, and there must be at least one. The first is a control word of 12 bits in hexadecimal and the second is a decimal cycle count. As per the comments, the control word is split into 3 nibbles, with the first giving some simulation control to stop or finish the simulation (bits 3:2), good for setting a timeout, and a bit to force all display output, overriding any other display settings. The second nibble controls what level of detail is displayed, with flags for transaction, data link and PHY layer output. The top bit of the nibble turns on raw unformatted output. When multiple layers are enabled, the higher layers (with TLP highest and PHY lowest) will be indented an appropriate amount to make things easier to read and separate the information. The top nibble controls which part of the link is to be displayed, whether just the received packets or transmitted packets or both. When multiple *pcieVHosts* are instantiated, the EP or RC configured models can be controlled separately. So, bit 8 controls whether a model should decode its own transmitted output or not, and the next two bits (10:9) whether to display if an endpoint or if a root complex. By default, the output is coloured, but this can be disabled with bit 11 set—useful if wishing to redirect the output to a text file

The example shows that, initially, no output is to be displayed, corresponding to the link training period where a lot of display link output is generated. Link display output slows down the simulation, which is why it can be turned on and off at useful points for maximum simulation speed. Then at a (*pipeclk*) cycle count equal to 5892 (0x1704) all three layers are enabled, but not raw symbols, with root complex enabled (which *pcieVHost* is, in this test) and with colour output remaining enabled. A third line gives a timeout cycle a long way into the simulation. A fragment of the output from the test is shown below.

```
~/git/pcievhost/vhdl/testaltpcie
PCIED0: {STP
PCIED0: 00 2d 00 00 00 01 00 01 0d 0c 00 01 01 04 0b 2c 3f f2
PCIED0: END}
PCIED0: ...DL Sequence number=45
PCIED0: .....TL Mem read req Addr=00010104 (32) RID=0001 TAG=0d FBE=1100 LBE=0000 Len=001
PCIED0: .....Traffic Class=0, Strong ordering (PCI)
PCIED0: .....TL No ECRC
PCIED0: ...DL Good LCRC (0b2c3ff2)
PCIED0: {SDP
PCIED0: 00 00 00 2a fb bc
PCIED0: END}
PCIED0: ...DL Ack seq 42
PCIED0: ...DL Good DLLP CRC (fbbc)
PCIEU1 00: PL Skip ordered set
PCIEU1 00: PL Skip ordered set
PCIEU1: {SDP
PCIEU1: 90 19 00 18 d1 9b
PCIEU1: END}
PCIEU1: ...DL UpdateFC-NP VC0 HdrFC=100 DataFC=24
PCIEU1: ...DL Good DLLP CRC (d19b)
PCIEU1: {SDP
PCIEU1: 00 00 00 2d 9c fe
PCIEU1: END}
PCIEU1: ...DL Ack seq 45
PCIEU1: ...DL Good DLLP CRC (9cfe)
PCIEU1: {STP
PCIEU1: 00 2b 4a 00 00 01 00 00 02 00 01 0d 06 00 00 fe ca 2f 5e b8 b1
PCIEU1: END}
PCIEU1: ...DL Sequence number=43
PCIEU1: .....TL Completion with data Successful CID=0000 BCM=0 Byte Count=002 RID=0001 TAG=0d Lower Addr=06
PCIEU1: .....Traffic Class=0, Strong ordering (PCI), Payload Length=0x001 DW
PCIEU1: .....0000fecafca
PCIEU1: .....TL No ECRC
PCIEU1: ...DL Good LCRC (2f5eb8b1)
```

Here we see the down link (from RC to EP) labelled PCIED0 and coloured green, separated from the up link (EP to RC) labelled PCIEU1 and coloured blue. Starting from the top three lines (for the down link) displayed is the decoded symbols on the PHY layer for a memory read TLP, bracketed by the STP and END symbols. Indented from this is the data link layer start, showing the sequence number for the DLLP layer packet. Indented from this is the TLP, decoded and formatted for the read request. The DLL LCRC then follows this, indented back to the DLL level. DLL layer flow control update and acknowledge packets are then seen returned from EP, before the memory read request completion, with a 32-bit word data payload.

Results

So, what do we get if we run the simulation? If we disable link display for this run in ContDisps.hex, the LTSSM code with the *pcieVHost* code outputs its change of state, and the test code prints out its progress reading and then setting the configuration space, before doing some memory writes and read-backs. The output then looks like the following (I've edited out the Altera PCIe model's output from its PLL models):

```
---> Detect Quiet (node 0)
---> Detect Active (node 0)
---> rcvr_idle_status = 0 (node 0)
---> Polling Active (node 0)
---> Polling Config (node 0)
---> Active lanes = 0x0001 (node 0)
---> Configuration Start (node 0)
---> Configuration Linkwidth Accept (node 0)
---> Configuration Lanenum Wait (node 0)
```

```

---> Configuration Lanenum Accept (node 0)
---> Configuration Complete (node 0)
---> Configuration Idle (node 0)
---> Configuration exit to L0 (node 0)
====> Vendor ID read back 0x14fc
====> Device ID read back 0x0001
====> Command read back 0x0000
====> Status read back 0x0010
====> Revision ID read back 0x01
====> Class code read back class=0x02 subclass=0x80 progif=0x01
====> Header type read back 0x00
====> BAR0 writing 0xffffffff
====> BAR0 read back 0xffffffff = 4096 bytes
====> BAR1 writing 0xffffffff
====> BAR1 read back 0x00000000 = 0 bytes
====> BAR2 writing 0xffffffff
====> BAR2 read back 0x00000000 = 0 bytes
====> BAR3 writing 0xffffffff
====> BAR3 read back 0x00000000 = 0 bytes
====> BAR4 writing 0xffffffff
====> BAR4 read back 0x00000000 = 0 bytes
====> BAR5 writing 0xffffffff
====> BAR5 read back 0x00000000 = 0 bytes
====> Found MSI capability
    0x008a7805
    0x00000000
    0x00000000
    0x00000000
    0x00000000
    0x00000000
====> Found Power Mangement capability
    0x00038001
    0x00000000
====> Found PCIe capability
    0x00020010
    0x00008020
    0x00002810
    0x01406411
    0x10110000
    0x00040000
    0x00000000
    0x00000000
    0x00000000
    0x0000001f
    0x00000000
    0x00000000
    0x00010001
    0x00000000
    0x00000000
====> BAR0 setting address to 0x00010000
====> Writing 0x0002 to Command register to enable memory accesses
====> Writing word 0x90dc0de to mem addr 0x00010080
====> Writing half-word 0xcafe to mem addr 0x00010106
====> Read back word 0x90dc0de from mem addr 0x00010080
====> Read back half-word 0xcafe from mem addr 0x00010106

***** test finished with no errors: PASS *****

```

I've highlighted a couple of things to cross reference back to the original IP configuration done at the start of this document in part 1.

And there we have it—the original goals have now been met and we have a test environment with which FPGA logic development can now take place, verifying its integration with the Altera PCIe IP and being driven from *pcieVHost*.

Conclusions

Over the course of the two parts of the document we've gone from a cold start to having a working test environment that can drive third-party commercially available PCIe IP and connect to user application logic that wishes to use the interface, allowing memory mapped transactions with the IP being developed. Necessarily, for clarity, the case study has made choices based on the minimal, but realistic, settings and configurations. None-the-less, the capabilities of the PCIe IP have been discussed, such as wider link widths, root complex instead of endpoint, and full 8b10b encoded serial links in place of PIPE interface. The *pcieVHost* VIP can be configured for all of these scenarios and can run the same test code for each setting.

Most of the test examples in the *pcieVHost* repository have, until now, been using the model back-to-back, with one configured as a root complex, and one as an endpoint. The new test bench using the Altera PCIe IP for Cyclone V, and this document that discusses it, is meant to serve as a more realistic test case for setting up a logic simulation test environment with real third party IP where FPGA logic development and verification can be done within an R&D lab. This also maps to an ASIC development, with an example of this mentioned in the first part, and to other FPGA vendors that offer a PCIe solution, whether soft or hard. The *pcieVHost* model's capabilities also allow many more scenarios than can be outlined here, and more complex exception conditions can be generated than there was time for in this document, showing its flexibility and sophistication in driving a simulation with a PCIe interface.

Further Reading

Below are gathered all the links made in this document for ease of reference

Co-simulation VIP References

- [pcieVHost Verification IP](#) github repository
 - [pcieVHost documentation](#)
- [VProc virtual processor](#) github repository
 - [VProc documentation](#)
- [Memory Model](#) github repository
 - [Memory Model documentation](#)

Articles

- [PCIe Primer](#) article ([PDF version](#) all parts)

- Article on [A PCIe model in C \(PDF version\)](#)
- Article on [The VProc Virtual Processor Verification IP \(PDF version\)](#)
- Article on [Arbitrary Protocol Modelling Using the VProc Co-simulation IP \(PDF version\)](#)

External References

- [Intel FPGA Licensing Support Center](#)
- [Altera FPGA Software Download Center](#)
- [Cyclone V Hard IP for PCI Express User Guide](#)